

CHAP*2 : **Les types de données en MySQL**

I. INTRODUCTION :

Dans ce chapitre, toutes les informations en ce qui concerne les types de données sont discutées ici afin d'éclaircir toutes âmes et répondre ainsi aux nombreuses questions qui nous hantent jour et nuit. Une discussion très exhaustive du type numérique avec exemples sera aussi exposée.

En addition, les conditions à remplir pour un bon stockage de données afin de savoir comment faire le choix du type de données à utiliser pour une colonne seront aussi discutées ici.

OBJECTIFS DU COUR :

Les objectifs de cour sont bien claires, ça veut tout simplement dire qu'après avoir étudié ce chapitre, vous devez être à mesure de :

- décrire l'importance des types de données
- appliquer les types de données comme : Numeric, String, Date et Time.
- Décrire les conditions de remplissage des colonnes en MySQL
- Expliquer comment choisir le bon type de données pour une colonne.
- Expliquer comment utiliser le type d'une colonne appartenant à un mécanisme d'une base de données.

II. LES DIFFERENTS TYPES DE DONNEES:

MYSQL support un grand nombre de type de données dans des différentes catégories : **type numeric, type date et time, type string (caracteres) et spatial**. Ce chapitre donne une vue d'ensemble en ce qui concerne ces types de données puis des explications bien détaillées de chaque catégorie.

1. LE TYPE NUMERIC

La description de ce type de données suit les conventions ci-dessous :

- **M** indique la largeur maximale du nombre entier que peut posséder une colonne de type **integer** la valeur ici est de **255**.
- Pour le type **floating-point et fixed-point** (les nombres à virgule) **M** indique le nombre total des chiffres qui peuvent être enregistrés (du côté **entier juste**) et le **D** indique le nombre total des chiffres qui viennent après la virgule (le côté décimal uniquement) et la valeur maximale de **D** est 30.

Le type numeric comprend plusieurs sous types que nous allons mentionner ici :

- **SERIAL**
Serial est un nom d'emprunt (alias) du type **BIGINT** en **not null auto_increment** (auto incrément).
- **BIT [(M)]**

Ce type transforme la colonne en un champ de **bits (001111)**. Le **M** ici indique le nombre de **bits** que pourra une **valeur** et cela varie entre **1-64bits** par valeur.

- **TINYINT [(M)]**

C'est la catégorie des plus petits nombres de la famille des **integers**. Quand ils sont signés, ils rangent entre **-128 à 127**. Et quand ils sont non-signés, **ils rangent** entre **0 à 255**.

- **BOOL, BOOLEAN :**

Ces types sont les synonymes des types **TINYINT(1)**. Ou la valeur de zéro est considérée comme étant fausse et celle différente de zéro est considérée comme étant vraie.

- **SMALLINT [(M)]**

Sont petits nombres de la famille des **integers**. Quand ils sont signés, ils rangent entre **-32768 à 32767**. Et quand ils sont non-signés, **ils rangent** entre **0 à 65535**.

- **MEDIUMINT [(M)]**

Est un petit nombre de la famille des **integers** mais de taille **medium**. Quand ils sont signés, ils rangent entre **-8388608 à 8388607**. Et quand ils sont non-signés, **ils rangent** entre **0 à 167772155**.

- **INT [(M)]**

Ceux-ci sont de la taille normale des nombres appartenant à la famille des **integers**. Quand ils sont signés, ils rangent entre **-2147483648 à 2147483647**. Et quand ils sont non-signés, **ils rangent** entre **0 à 4294967295**.

- **INTEGER [(M)]**

Ce type est le synonyme du type INT, souvent légèrement plus grand qu'INT.

- **BIGINT [(M)]**

Ceux-ci sont de la taille plus grande que les **integers**. Quand ils sont signés, ils rangent entre **-9223372036854775808 à 9223372036854775807**. Et quand ils sont non-signés, **ils rangent** entre **0 à 18446744073709551615**.

NB : toutes opérations arithmétiques sont faites à partir de **BIGINT** ou **DOUBLE**.

- **FLOAT [(M, D)]**

Ceux-ci sont très petits nombres de la famille des décimaux. Quand ils sont signés, ils rangent entre **-3.402823466^{E+38} à -1.175494351^{E-38,0}** Et quand ils sont non-signés, **ils rangent** entre **1.175494351^{E-38} à 3.402823466^{E+38}**.

- **DOUBLE [(M, D)]**

Double est la taille normale des nombres a virgule. Les valeurs permises sont comme suites : **-1.7976931348623157^{E+308} à -2.2250738585072014^{E-308}, 0, et 2.2250738585072014^{E-308} à 1.7976931348623157^{E+308}**.

- **DECIMAL [(M [, D])]**

Décimal est l'exacte point fixe des nombres décimaux. M est le nombre total des chiffres se trouvant sur le côté entier (la précision) et D est le nombre total des chiffres se trouvant du côté décimal, après la virgule. Dans ce type décimal, les nombres négatifs ne sont pas permis. Voilà pourquoi il est souvent utilisé pour les sommes d'argent (par exemple : 123.56\$). Le nombre maximal que M peut contenir est de 65 et D est de 30. La valeur par défaut de D est 0 celle de M est 10.

2. **LE TYPE STRING**

Les types string sont : CHAR, VARCHAR, BINARY, VARBINARY, BLOB, TEXT, ENUM, et SET.

- **LE TYPE CHAR ET VARCHAR**

Ces deux types sont très similaires, très populaire mais présentent aussi plusieurs points différents.

La longueur de caractères par exemple, CHAR est très stricte en terme de longueur car elle demeure fixe et stricte depuis la création du tableau et elle peut être n'importe qu'elle chiffre allant de 0 à 255. Lorsqu'une valeur en CHAR est chargée, elle prend ne pas en considération les espaces, elle les élimine carrément. Par contre avec VARCHAR, la longueur de la colonne est variable et peut prendre une longueur de valeur allant de 0 à 65535. VARCHAR n'accepte pas que les données en texte, mais aussi les nombres et quelque caractères spéciaux. Très souvent, VARCHAR est beaucoup conseillé à cause de sa flexibilité et ses multiples fonctions.

- **LE TYPE BLOB ET TEXT**

Le type blob, est le large objet binaire qui peut contenir un montant variable de données. Les quatre sous types du type blob sont : TINYBLOB, BLOB, MEDIUMBLOB, et LONGBLOB. Leurs différences se situent juste au niveau de la longueur maximale de valeur que peut contenir chacun.

Les quatre sous types du type TEXT sont : TINYTEXT, TEXT, MEDIUMTEXT, et LONGTEXT. Ceux-ci correspondent exactement aux quatre sous types du type blob et ont la même valeur maximale et les mêmes conditions requises pour le stockage. Dans une colonne blob, les données sont traitées en binaire (byte ou octet) par contre une colonne de type TEXT est traitée comme des données non-binaires et a un ensemble de caractère (caractères compréhensibles à l'œil nu) par contre le type blob n'a pas d'ensemble de caractères (caractères incompréhensibles à l'œil nu parce que le blob traite les informations en bytes.).

- **LE TYPE ENUM**

Le type **enum** (énuméré) est un objet, d'ensemble de caractères (**ligne de caractères** ou **des mots**) avec une valeur choisie parmi la liste des valeurs autorisées et énumérées explicitement dans une colonne lors de la création du tableau.

Par exemple : pour créer un tableau ayant une colonne **enum**, voici comment s'y prendre. Voir l'image ci-dessous :

```
c:\wamp\bin\mysql\mysql5.5.8\bin\mysql.exe
mysql> create table mesure(nom enum('petit', 'moyen', 'grand'));
Query OK, 0 rows affected (0.38 sec)

mysql>
```

Fig.1. création d'un tableau avec une colonne **enum** et la liste des **valeurs**.

```
c:\wamp\bin\mysql\mysql5.5.8\bin\mysql.exe
mysql> desc mesure;
+-----+-----+-----+-----+-----+-----+
| Field | Type                               | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| nom   | enum('petit','moyen','grand')      | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.30 sec)

mysql>
```

Fig.2. description du tableau mesure.

NB : Que les valeurs définies parmi la liste des valeurs énumérées peuvent être insérées dans la colonne **nom** qui est de type **enum**. Et on ne peut qu'insérer une seule valeur à l'instant.

• LE TYPE SET

Le type SET est aussi un objet d'ensemble de caractères (ligne de caractères (string)) à la différence du type enum, SET peut accepter de zéro à un nombre indéfini des valeurs venant d'une liste de valeurs prédéfinies. Les valeurs se définissent dès le commencement de la création du tableau. Lors de l'insertion des données, il est possible d'y insérer plus qu'une valeur à l'instant parmi celles qui sont prédéfinies, séparées par une virgule car SET n'accepte pas les espaces, les duplications et le maximum des différentes valeurs à insérer est de **64**. Voir les images ci-dessous pour plus de compréhensions :

```
c:\wamp\bin\mysql\mysql5.5.8\bin\mysql.exe
mysql> create table restau_cmd(nom_pizza set('jambon','fromage','tomate','poulet',
', 'poivron'));
Query OK, 0 rows affected (0.11 sec)

mysql>
```

Fig.3.creation d'un tableau avec une colonne set contenant des valeurs différentes.

```
c:\wamp\bin\mysql\mysql5.5.8\bin\mysql.exe
mysql> insert into restau_cmd (nom_pizza) values ('fromage');
Query OK, 1 row affected (0.09 sec)

mysql> insert into restau_cmd (nom_pizza) values ('fromage,jambon');
Query OK, 1 row affected (0.30 sec)

mysql>
```

Fig.4. insertion des données dans le tableau à colonne de type SET.

```

c:\wamp\bin\mysql\mysql5.5.8\bin\mysql.exe
mysql> select * from restau_cmd;
+-----+
| nom_pizza |
+-----+
| fromage  |
| jambon,fromage |
+-----+
2 rows in set (0.00 sec)

mysql>

```

Fig.5. visualisation du tableau.

NB : il ne faut surtout pas chercher à insérer des valeurs ne font pas parties de la liste des prédéfinies a risque de déclencher des espaces vides et aucune présence de la nouvelle valeur comme l'indique l'image ci-dessous :

```

c:\wamp\bin\mysql\mysql5.5.8\bin\mysql.exe
mysql> insert into restau_cmd (nom_pizza) values ('viande');
Query OK, 1 row affected, 1 warning (0.08 sec)

mysql>

```

Fig.6. insertion de la valeur hors de la liste avec succès.

```

c:\wamp\bin\mysql\mysql5.5.8\bin\mysql.exe
mysql> select * from restau_cmd;
+-----+
| nom_pizza |
+-----+
| fromage  |
| jambon,fromage |
|          |
+-----+
4 rows in set (0.00 sec)

mysql>

```

Fig.7. présence d'espace mais pas de valeurs.

3. LE TYPE DATE ET TIME

Les différents sous types de DATE et TIME sont comme suit : DATETIME, DATE, TIMESTAMP, TIME, et YEAR. Tous ces sous types nous permettront représenter périodiquement les informations d'une manière très particulière selon le type choisi. MySQL représente les dates sous le format suivant : YYYY-MM-DD (ANNE-MOIS-JOUR) comme le décrit le tableau ci-dessous :

Type de date	Format (en valeur 0)
DATETIME	0000-00-00 / 00 :00 :00'
DATE	0000-00-00
TIMESTAMP	0000-00-00 / 00 :00 :00'
TIME	00 :00 :00
YEAR	0000

- **LE TYPE DATETIME, DATE, ET TIMESTAMP**

Les trois types sont très liés. Cette section explique leurs similitudes et les points qui les diffèrent.

Le datetime est utilisé lorsque vous avez besoin des informations à la fois de la date et du temps. MySQL affiche les informations de datetime sous ce format : AAAA-MM-JJ HH :MM :SS (année-mois-jour / heure : min : sec). La rangée supportée par ce type est de **1000-01-01 00 :00 :00 à 9999-12-31 23 :59 :59**.

Le type date est utilisé lorsque vous avez besoin que de la valeur de la Date uniquement. MySQL affiche les informations de date sous ce format : AAAA-MM-JJ (année-mois-jour). La rangée supportée par ce type est de **1000-01-01 à 9999-12-31**.

Le type timestamp est aussi utilisé lorsque vous avez besoin des informations à la fois de la date et du temps. MySQL affiche les informations de datetime sous ce format : AAAA-MM-JJ HH :MM :SS (année-mois-jour / heure : min : sec). La rangée supportée par ce type est de **1970-01-09 00 :00 :01 UTC a 2038-01-09 03 :14 :07 UTC**. Ce type a des propriétés variantes, qui dépendent à la version du logiciel MySQL que vous utilisez. Lors du stockage, les données sont converties du temps actuel à l'UTC et lors de la sortie l'opération s'inverse. Et cela se fait qu'avec timestamp.

- **LE TYPE TIME**

MySQL affiche le temps sous ce format HH :MM :SS (heures : minutes : secondes). La valeur du temps peut être rangé de **'-838 :59 :59' a '838 :59 :59'**. La partie heur peut être plus grande que les autres parce que le temps est utilisé pas seule pour représenter l'heur de la journée (qui est généralement plus petit que 24h) mais aussi l'intervalle de temps écoulé entre deux évènements (qui peut être plus large que 24h).

- **LE TYPE YEAR**

Le type year est utilisé pour représenter les années et peut être déclaré year (2) par exemple : 98 qui tient de 1998 et year (4) qui peut être 1976 qui est le format par défaut. Pour le format year (4) MySQL affiche les années en format de quatre caractères comme ceci : YYYY et qui a une rangée de 1901 à 2155 mais celui de year (2), il est rangé de 70 (1970) a 69 (2069).

III. CHOISIR LE BON TYPE POUR UNE COLONNE

Pour une bonne utilisation efficace de stockage, essayer de toujours utiliser le type le plus précis pour chaque cas. Par exemple : si une colonne integer sera utilisé pour des valeurs allant 1 à 99999, MEDIUMINT est le bon type pour ce cas. Pour une bonne représentation monétaire, le type DECIMAL est le plus conseillé. Car cela sera stocké en forme de texte afin de ne pas perdre l'exactitude décimale. Maintenant lorsque l'exactitude importe peu, le type DOUBLE peut aussi être utilisé à la place.

IV. RESUME

Dans ce chapitre, nous avons eu à étudier les points ci-dessous :

- Le type NUMERIC
- Le type STRING
- Le type DATE et TIME
- Comment choisir le bon type de données pour une colonne.

V. QUESTONS TERMINALES

Voici donc les questions terminales R2

- Décrivez les types float et double
- Enumérez les différences entre les types char et varchar en MySQL
- Citez les différents sous types de DATE et TIME avec leurs rangés d'intervalle.